

建構基本訊息樣式應用程式

來源：<https://codelabs.developers.google.com/tflite-nlp-basic?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將學習如何建構內含文字方塊和傳送按鈕的簡易訊息應用程式。

目錄

1. 事前準備
2. 在 Android Studio 中建立新專案
3. 建立使用者介面
4. 連結控制項並建立應用程式
5. 在 iOS 上建立 Basic 應用程式
6. 編輯分鏡腳本
7. 完成基本 iOS 應用程式的程式碼編寫
8. 恭喜！

步驟 1 · 約 2 分鐘

1. 事前準備

這個程式碼研究室是「開始使用行動裝置文字分類」路徑的活動 #2。

在本程式碼研究室中，您將建構簡單的訊息應用程式。稍後，您會在課程中更新這個應用程式，加入機器學習模型，從應用程式的訊息中篩除垃圾內容。

必要條件

- 具備使用 Kotlin 進行 Android 開發作業的基本知識，以及 (選用) 使用 Swift 進行 iOS 開發作業的基本知識

建構項目

- 簡單的訊息應用程式

軟硬體需求

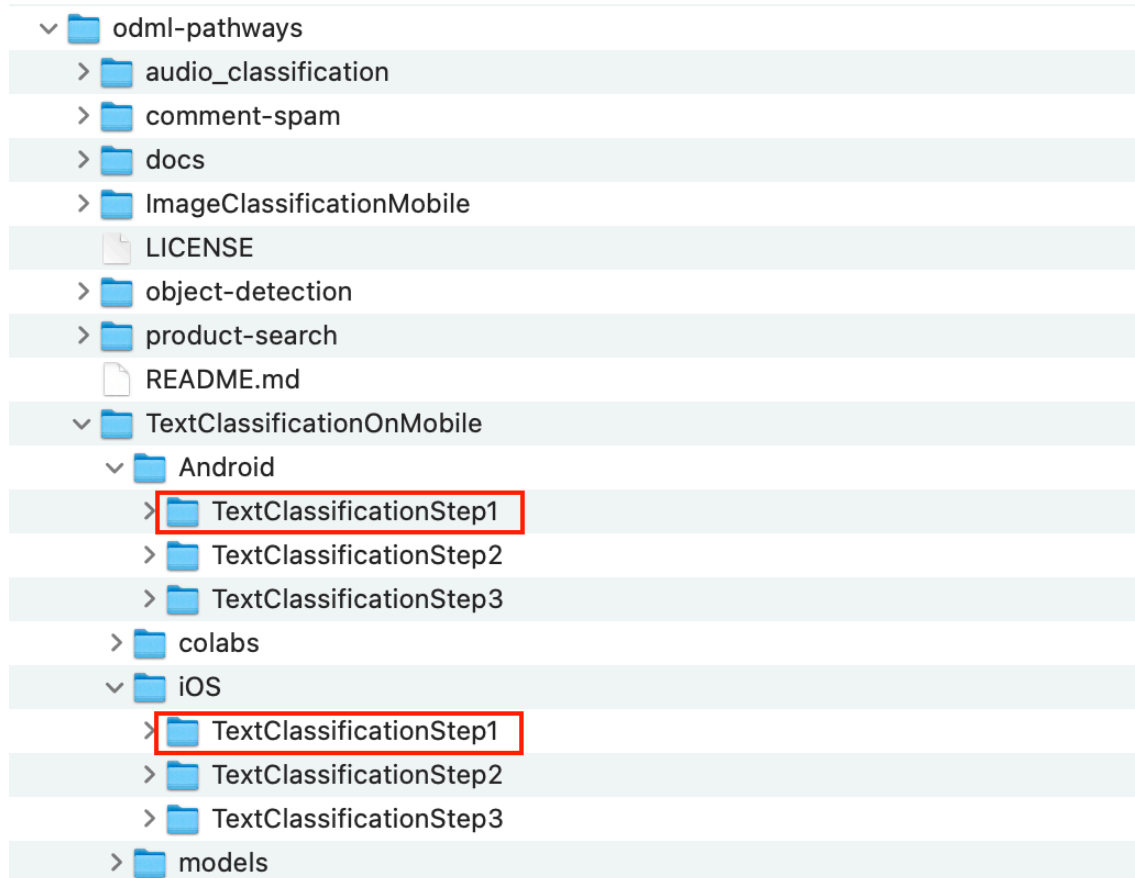
- 如要開始使用 Android，請先安裝 [Android Studio](#)。請先確認已安裝並完全更新，再繼續操作。
- 如果是 iOS，則需要 Xcode。你可以在 App Store 中找到這項應用程式。(如果只想編寫 iOS 應用程式，請直接跳到步驟 5。

取得程式碼

如果您不想逐步操作，只想查看這個路徑的最終程式碼，請前往

```
git clone https://github.com/googlecode labs/odml-pathways
```

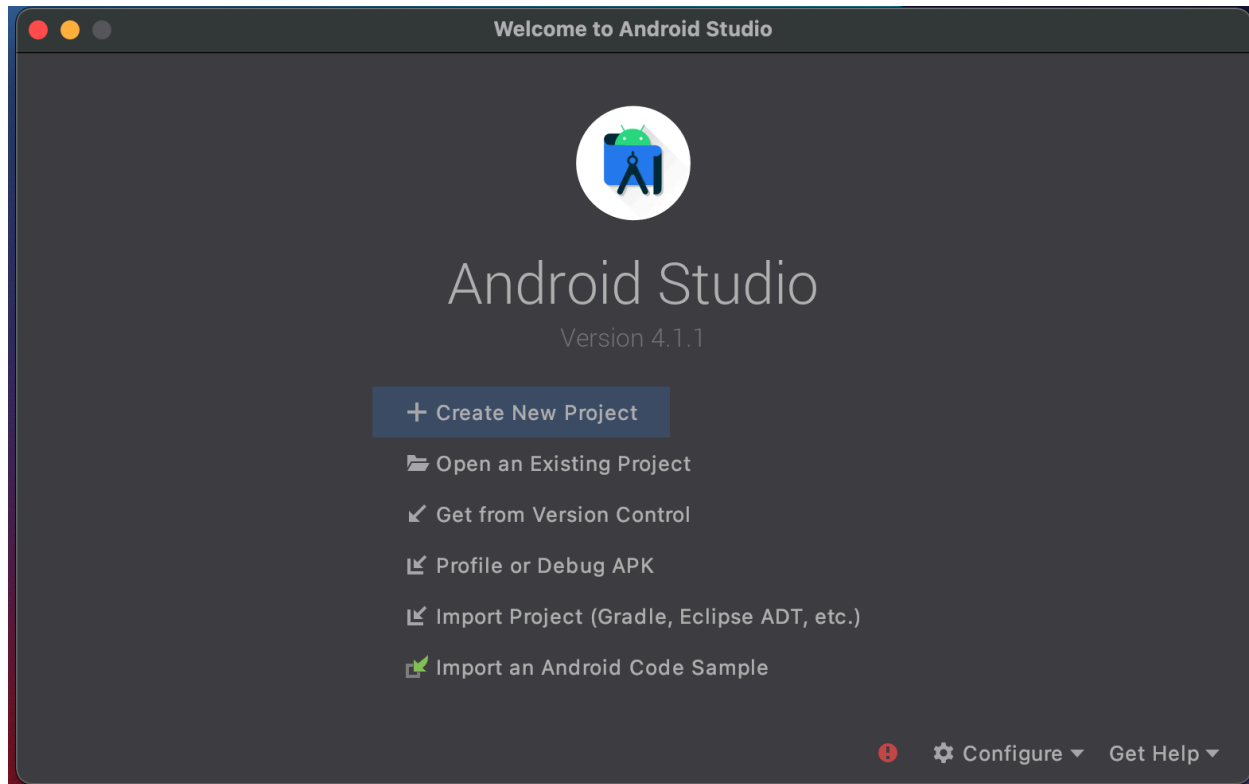
在當中找到 TextClassificationOnMobile 目錄，然後您會看到 Android 和 iOS 子目錄。這些目錄會包含 TextClassificationStep1 子目錄，其中分別含有 Android 和 iOS 上的應用程式。



步驟 2 · 約 2 分鐘

2. 在 Android Studio 中建立新專案

1. 啟動 Android Studio。



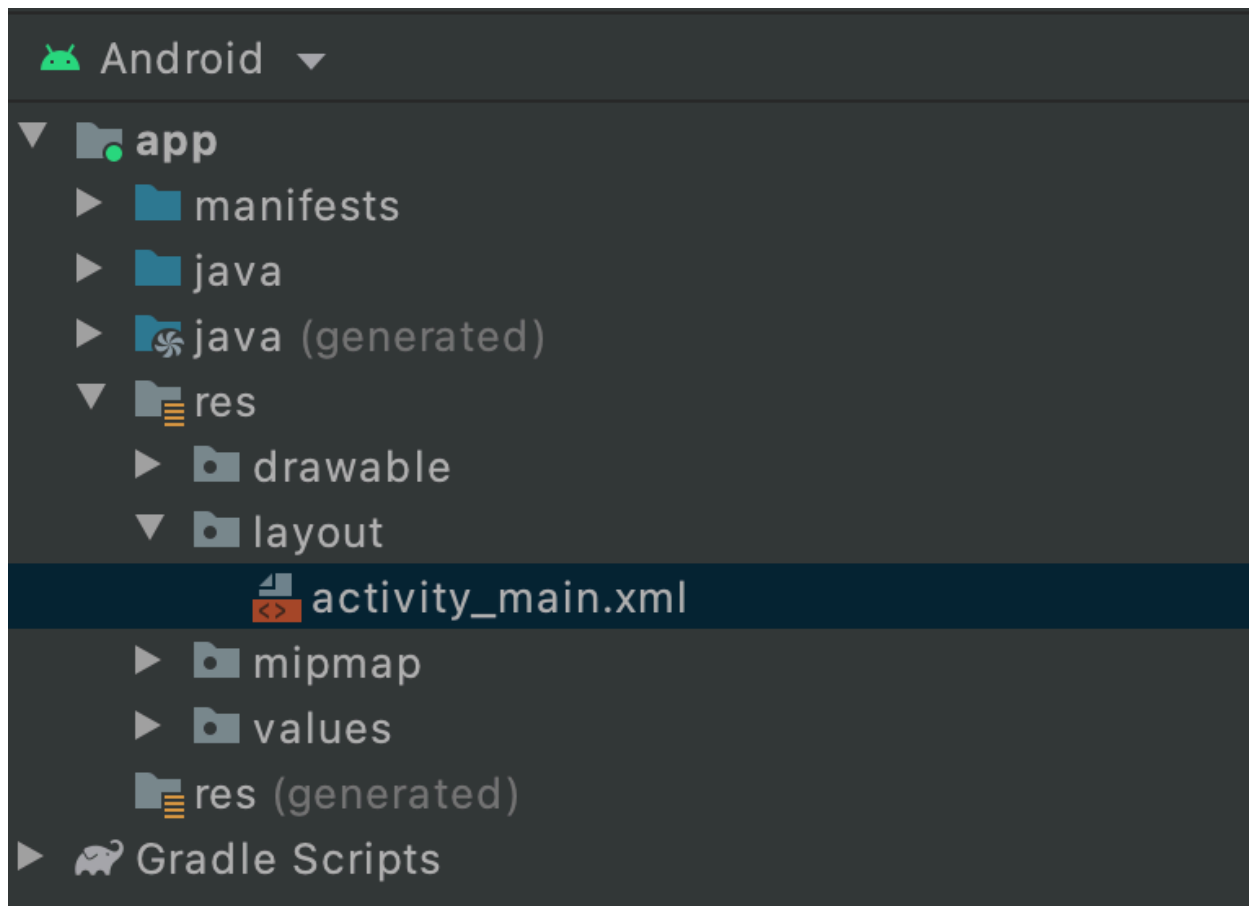
1. 選取「建立新專案」。畫面上會顯示對話方塊，要求您選取專案範本。
 2. 選取「Empty Activity」，然後點選「Next」。接著，系統會要求您設定專案。您可以選擇任何項目，但請務必將語言設為 **Kotlin**，並將最低 SDK 設為 **API 23**。
 3. 按一下「完成」。完成後，Android Studio 會開啟專案。Android Studio 可能需要幾分鐘執行 Gradle 同步處理，確保一切正常運作，尤其是您第一次使用 Android Studio 時。
-

步驟 3 · 約 3 分鐘

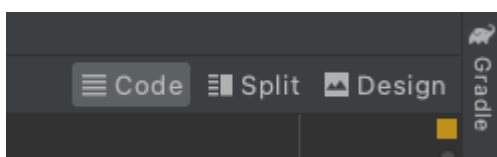
3. 建立使用者介面

Android 應用程式的使用者介面是使用名為版面配置檔案的 XML 檔案建立。

1. 開啟檔案。在 Android Studio 的檔案總管中，依序點選「app」>「res」>「layout」>「activity_main.xml」。



畫面的右上角應該會顯示「程式碼」、「分割」和「設計」的標籤，如下所示：



請務必先選取「程式碼」，再繼續下一個步驟。

2. 將程式碼替換為下列 XML：

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <LinearLayout android:orientation="vertical"
        android:layout_height="match_parent"
        android:layout_width="match_parent">
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">
            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Enter a string:"></TextView>
            <EditText
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:id="@+id/txtInput"></EditText>

        </LinearLayout>

        <TextView
            android:id="@+id/txtOutput"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Output Goes here"
            android:textSize="36sp"></TextView>
        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:id="@+id/btnSendText"
            android:text="Send"></Button>

    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>

```

這會提供基本的使用者介面，內含 `TextView` 的輸入區域，提示您「Enter a String」(輸入字串)，以及可用於輸入字串的 `EditText` (名為「txtInput」)。下方是 `TextView`，用於算繪輸出內容，以及使用者按下即可觸發分類的按鈕。

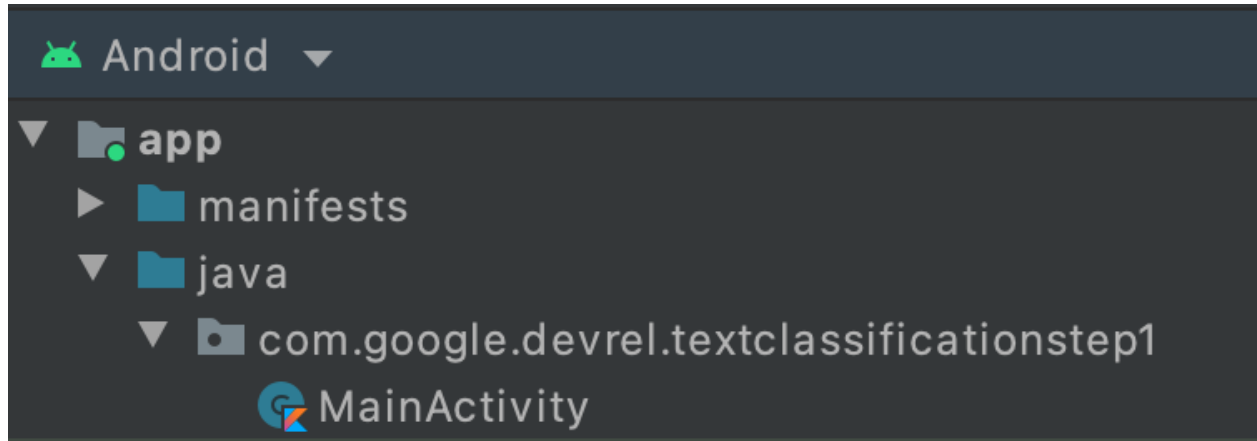
下一步是編寫程式碼，啟動這個使用者介面。

步驟 4 · 約 3 分鐘

4. 連結控制項並建立應用程式

如要編輯這項活動的程式碼，請找出 MainActivity 程式碼檔案。

1. 在 Android Studio 中，依序點選「app」>「java」>「MainActivity」。



2. 開啟 MainActivity 檔案，前往程式碼編輯器。將這個檔案中的所有內容(除了以「package」開頭的第一行) 替換為下列程式碼：

```
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView

class MainActivity : AppCompatActivity() {
    lateinit var txtInput: EditText
    lateinit var btnSendText: Button
    lateinit var txtOutput: TextView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        txtInput = findViewById(R.id.txtInput)
        txtOutput = findViewById(R.id.txtOutput)
        btnSendText = findViewById(R.id.btnSendText)
        btnSendText.setOnClickListener {
            var toSend:String = txtInput.text.toString()
            txtOutput.text = toSend
        }
    }
}
```

這段程式碼會將版面配置上的控制項連結至 `txtInput`、`txtOutput` 和 `btnSendText`，以便在程式碼中處理這些控制項。接著，系統會為按鈕設定 `OnClickListener`，這樣一來，使用者觸控按鈕時，系統就會讀取 `txtInput` 中的文字、轉換為字串，然後將 `txtOutput.text` 屬性設為該字串。

如果您正在學習「開始使用行動裝置文字分類」課程，這就是您在 Activity #1 (示範影片) 中看到的行為，也就是應用程式只會使用使用者輸入的內容，不會檢查垃圾內容！

在下一個程式碼研究室中，您將瞭解如何在應用程式中使用機器學習模型，篩除垃圾留言。

如要在 iOS 上建構相同應用程式，請繼續進行本程式碼研究室。

步驟 5 · 約 5 分鐘

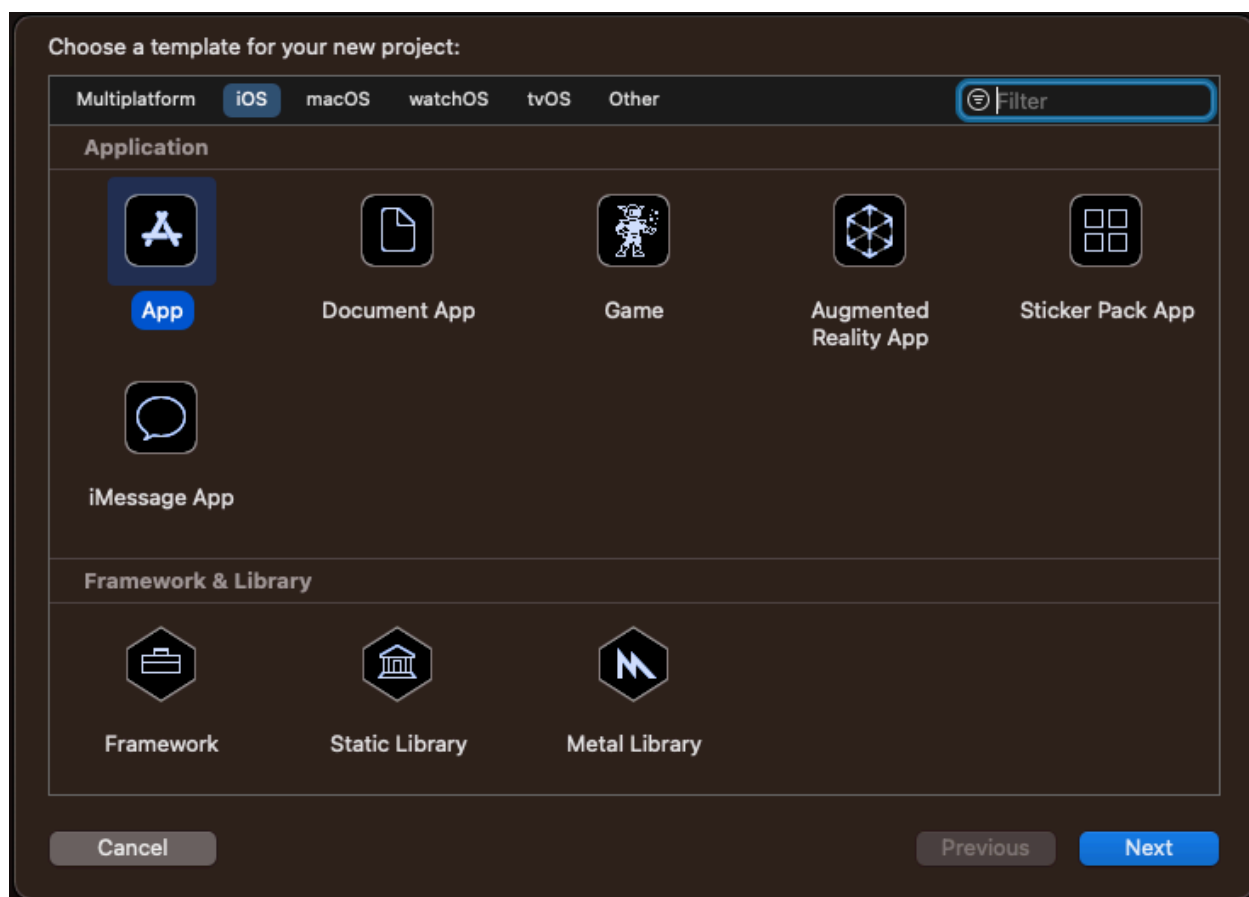
5. 在 iOS 上建立 Basic 應用程式

如要完成下列步驟，您必須熟悉 Xcode 和使用 Swift 進行 iOS 開發。

如果不想完成建立程序，可以複製存放區並直接開啟應用程式。這個步驟稱為 TextClassificationStep1，位於存放區的 iOS 資料夾中。

如要建立基本應用程式，請先使用 Xcode。

1. 使用基本範本建立新的應用程式：



2. 為新專案選擇選項。為產品提供名稱和機構 ID。您可以輸入所需內容，或按照下列範例操作：(但請務必如圖所示，將介面設為「Storyboard」，生命週期設為「UIKitApp Delegate」)。

Choose options for your new project:

Product Name: TextClassificationStep1

Team: None

Organization Identifier: com.google.devrel

Bundle Identifier: com.google.devrel.TextClassificationStep1

Interface: Storyboard

Life Cycle: UIKit App Delegate

Language: Swift

☐ Use Core Data

☐ Host in CloudKit

☒ Include Tests

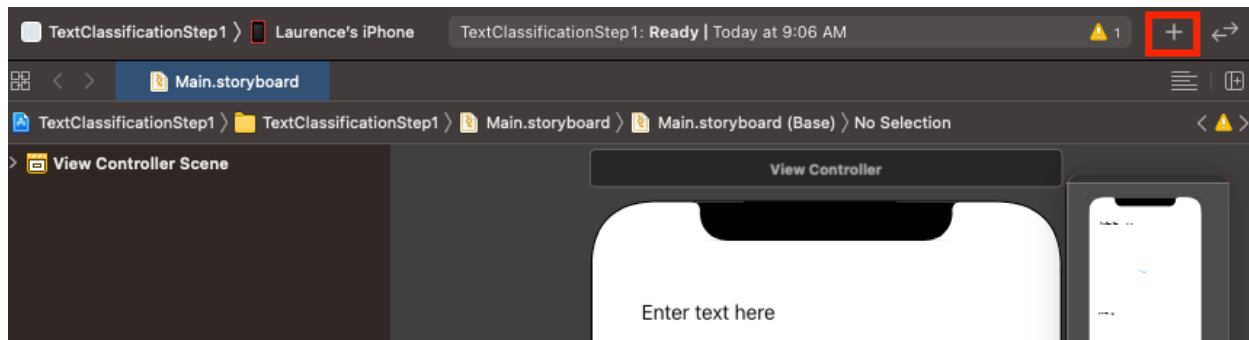
Cancel

Previous

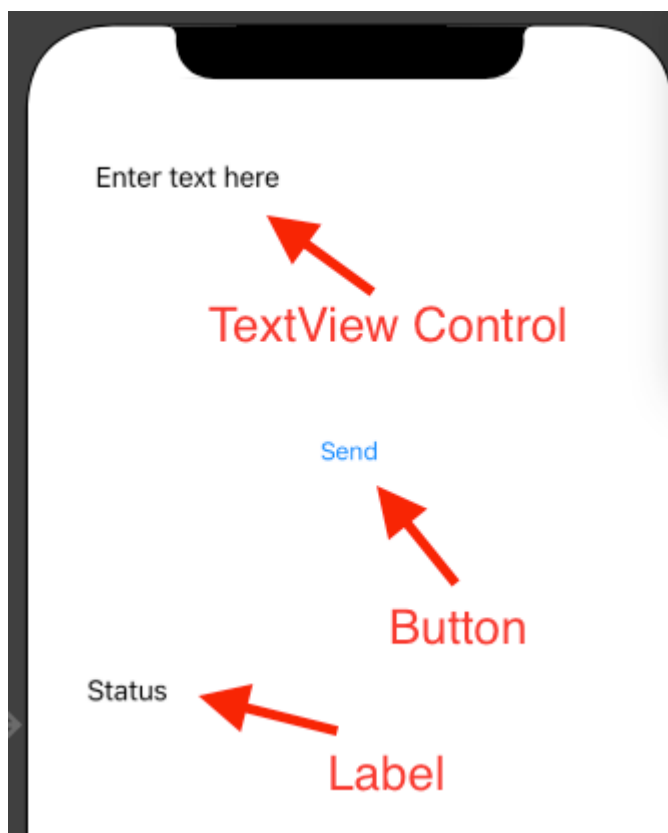
Next

6. 編輯分鏡腳本

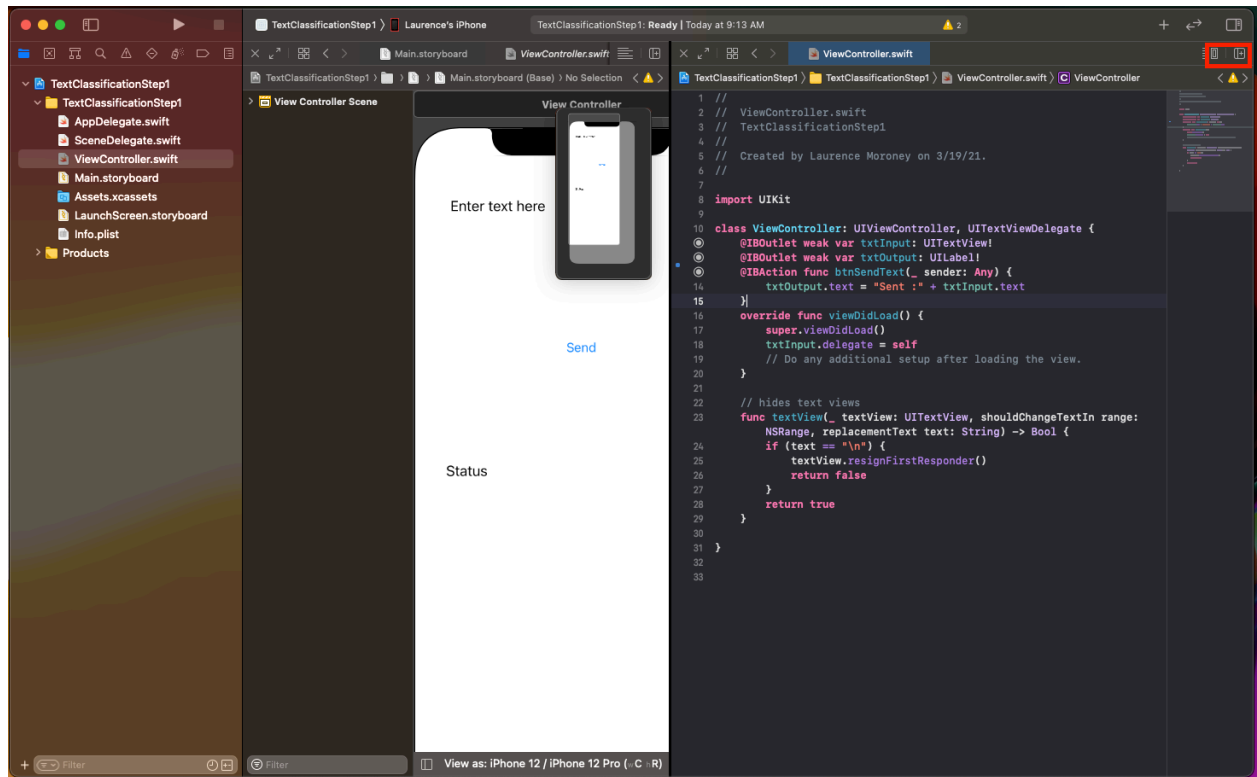
1. 開啟「Main.storyboard」 **Main.storyboard**。畫面上會顯示設計介面，您可以在其中新增控制項。
2. 如要新增控制項，請按一下 Xcode 視窗頂端的「+」按鈕。



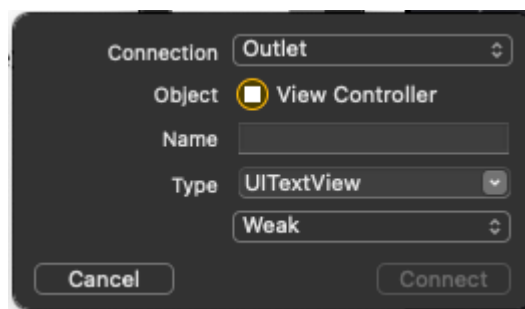
3. 您可以使用這個工具，將 TextView 控制項、Button 控制項和 Label 控制項放在設計介面上。如下所示：



4. 使用輔助程式開啟並排檢視畫面，同時開啟腳本和 **ViewController.swift** 檔案。助理是畫面右上角的小圖示，如下所示：



1. 按下 CONTROL 鍵，然後將 TextView 控制項拖曳到程式碼介面。拖曳時，畫面上應該會顯示藍色箭頭。
2. 將程式碼拖曳到類別宣告正下方的程式碼上，系統會顯示彈出式視窗，詢問連線類型。如下所示：



3. 選取「Outlet」(出口)，並將其命名為 txtInput。
4. 對標籤執行相同操作，然後將連線類型設為「Outlet」，並命名為 txtOutput。
5. 最後，將按鈕拖曳到畫面上，但這次請選取「動作」做為連線類型。(請勿使用「插座」做為連線類型)。
6. 將動作命名為 **btnSendText**。

完成後，類別頂端的程式碼應如下所示：

```
@IBOutlet weak var txtInput: UITextView!  
@IBOutlet weak var txtOutput: UILabel!  
@IBAction func btnSendText(_ sender: Any) {  
}
```

7. 完成基本 iOS 應用程式的程式碼編寫

由於這個檢視區塊使用 UITextView，如要讓它回應檢視區塊上的事件，就必須是 UITextViewDelegate。

如要取得這項資訊，請將類別宣告變更為：

```
class ViewController: UIViewController, UITextViewDelegate {
```

接著，在 viewDidLoad 函式中，將 UITextView (設定插座時稱為 txtInput) 的委派項目設為這個類別，如下所示：

```
override func viewDidLoad() {
    super.viewDidLoad()
    txtInput.delegate = self
}
```

最後，您希望在使用者按下鍵盤上的 Enter 鍵後，處理移除鍵盤顯示畫面的問題。這是透過您剛才設定的委派完成，您可以在 ViewController 上實作這個方法來處理。

```
func textView(_ textView: UITextView, shouldChangeTextIn range:
    NSRange, replacementText text: String) -> Bool {
    if (text == "\n") {
        textView.resignFirstResponder()
        return false
    }
    return true
}
```

這個應用程式的按鈕動作很簡單，我們只會將使用者輸入的內容傳遞至輸出內容。稍後您會瞭解如何使用自然語言處理模型篩選這段文字。但目前先將其接線，模擬直通。

您已建立動作 btnSendText，現在只需要將程式碼新增至該動作即可：

```
@IBAction func btnSendText(_ sender: Any) {
    txtOutput.text = "Sent :" + txtInput.text
}
```

步驟 8 · 約 1 分鐘

8. 恭喜！

您已完成本程式碼研究室！

在下一個學習路徑中，您將瞭解如何建立自然語言處理模型，稍後再將模型帶回這個應用程式，讓應用程式能夠篩選使用者文字中的垃圾留言！
